

5회차 ML 과제

2022147009 정호진

실제 신용카드 사기 데이터셋을 활용해 클래스 불균형 상황에서 분류 모델 학습

1. 데이터 로드 및 기본 탐색

- 신용카드 사기 데이터셋
- Class 0: 정상 거래, Class 1: 사기 거래
- Time, Amount, Class 의 컬럼 가짐
- 나머지 V1 ~ V28 column은 PCA로 압축된 데이터 -> 의미 파악 불가
- 전체 284,807개의 row, 31개의 column
- 모든 column non - null
- 분류 목적인 Class의 비율이 0.998 : 0.002 인 심각한 불균형 데이터

2. 샘플링

- 사기 거래(Class 1) 492건 그대로 유지
- 정상 거래(Class 0) 10,000건 무작위 샘플링
 - 재현성 유지를 위해 random_state는 42로 유지
- 샘플링 후 Class 비율 0.953 : 0.046 으로 불균형 완화
- Time 열 제거
 - Time: 첫 거래로부터 경과 초

- ◆ 예측 정보 없음 + 스케일이 지나치게 커서 이후 SMOTE 사용 시 거리 계산에 왜곡 일으킴

3. 학습 데이터, 테스트 데이터 분할

- 8 : 2 비율로 데이터 분할
- Stratify = y 옵션으로 클래스 비율 유지
 - 10,000건을 샘플링 하였다 하더라도, 여전히 클래스 불균형이 심하므로, 위와 같은 방법으로 클래스 비율을 유지하여야 함
- 재현성 유지를 위해 random_state는 42로 설정

4. 데이터 전처리

- Amount 변수만 StandardScaler로 표준화 -> 새로운 변수 Amount_Scaled로 대체
- 원본 Amount 제거
- 데이터 누수 방지
 - 전체 데이터로 StandardScaler 학습 시 test 데이터의 평균, 표준편차가 train 데이터에 반영될 수 있음
 - Train 데이터에는 fit_transform을 적용하고, test 데이터에는 transform만 적용

5. SMOTE

- 10,000건만 샘플링 하였더라도, 여전히 클래스 불균형 문제가 심함
- 모델의 학습, 추론 시 패턴 파악에 악영향 끼칠 가능성 높음

- 데이터 누수 방지: Train 데이터에만 SMOTE 이용해야 함
 - X_train, y_train에만 SMOTE 적용
- SMOTE는 유클리디안 거리 기반 kNN으로 이웃 찾아 데이터 증강
 - 개별 데이터의 스케일이 중요
 - ◆ Time 컬럼 제거

6. 적합한 모델 선택 후 학습, Baseline 설정

- Metric
 - Classification report 로 precision, recall, f1 – score 확인
 - Average precision score로 pr – auc 계산
- Baseline
 - 튜닝되지 않은 rf, 로지스틱 회귀 등으로 baseline 설정
 - 로지스틱 회귀 / RandomForest / GradientBoosting / LightGBM / XGBoost 5종 비교
 - 모든 모델 random_state = 42로 고정하여 재현성 확보
- Baseline 결과 (Class 1 기준, test 2,099건 중 사기 98건)

[표 1] Baseline 모델별 성능 (Class 1)

모델	Precision	Recall	F1	PR-AUC	FP
LogisticRegression	0.6129	0.9694	0.7510	0.9557	60
RandomForest	0.9886	0.8878	0.9355	0.9479	1
GradientBoosting	0.7460	0.9592	0.8393	0.9585	32
LightGBM	0.8969	0.8878	0.8923	0.9461	10
XGBoost	0.8878	0.8878	0.8878	0.9481	11

- 결과 해석

- 로지스틱 회귀, GradientBoosting: Recall은 0.96 ~ 0.97로 높으나 Precision이 0.61 ~ 0.75로 낮음
 - ◆ FP가 각각 60건, 32건으로 많아 F1 목표(0.88) 미달
 - ◆ SMOTE로 소수 클래스를 20배 증폭한 결과, 결정 경계가 사기 쪽으로 과도하게 이동한 것으로 판단
 - RandomForest: Precision 0.9886(FP 1건), Recall 0.8878, F1 0.9355로 가장 우수
 - LightGBM, XGBoost: TP는 RF와 동일한 87건이나 FP가 10 ~ 11건으로 많아 F1이 0.89대에 머무름
 - PR-AUC는 5개 모델 모두 0.946 ~ 0.959로 큰 차이 없음
 - ◆ threshold와 무관한 확률 랭킹 능력 자체는 유사하며, 성능 차이는 threshold 0.5 기준의 판정 위치에서 발생
- 모델 선정: RandomForest
- Precision, F1 모두 최고이며 Recall도 목표(0.80)를 충족

7. 최종 성능 평가

- 최종 모델: RandomForest (기본 하이퍼파라미터, threshold = 0.5)
- 목표 달성 여부

[표 2] 최종 모델(RandomForest) 목표 달성 여부

지표	Class 0	Class 1	목표	달성
Recall	0.9995	0.8878	≥ 0.80	O
F1	0.9970	0.9355	≥ 0.88	O
PR-AUC	0.9986	0.9479	≥ 0.90	O

- Class 0, Class 1 모두 세 지표 전부 목표 충족
- 하이퍼파라미터 튜닝

- RandomizedSearchCV (n_iter = 20, cv = StratifiedKFold 5, scoring = average_precision)
- imblearn Pipeline으로 SMOTE를 CV 내부에 배치
 - ◆ X_train_smote를 그대로 투입하면 합성 샘플이 validation fold로 유입되어 CV 점수가 과대평가됨
 - ◆ 따라서 원본 X_train, y_train으로 fit하고 SMOTE는 각 fold의 학습 부분에만 적용
- 탐색 결과: n_estimators = 400, max_depth = 20, min_samples_leaf = 1, max_features = 'sqrt' / CV PR-AUC = 0.9239
- 해석: 탐색된 조합이 기본값과 실질적으로 동일
 - ◆ RandomForest 기본값은 n_estimators = 100, max_depth = None, min_samples_leaf = 1, max_features = 'sqrt'
 - ◆ min_samples_leaf, max_features는 기본값과 완전히 일치하며 나머지도 성능 차이를 만들지 못함
 - ◆ 즉 baseline이 이미 포화 상태이며 튜닝으로 얻을 개선이 없음
- CV PR-AUC(0.9239)와 test PR-AUC(0.9479)의 차이
 - ◆ 두 값은 직접 비교 대상이 아님
 - ◆ CV는 학습 데이터를 5분할하여 fold별 약 6,700건으로 학습한 5회 평균이므로 더 보수적인 추정치
 - ◆ test는 2,099건 단일 분할에 대한 결과
- Threshold 조정
 - precision_recall_curve로 전 구간의 F1을 계산한 뒤 최댓값 탐색
 - 최적 threshold = 0.520, F1 = 0.9355
 - ◆ 기본값 0.5의 F1 = 0.9355와 동일하여 개선 없음
 - 원인: Precision 개선 여지가 이미 소진된 상태
 - ◆ FP가 1건뿐이므로 Precision을 더 올릴 수 없음
 - ◆ Recall을 올리려면 threshold를 낮춰 FN 11건을 잡아야 하나, 그만큼 FP가 증가해 F1이 상쇄됨
 - ◆ 따라서 baseline이 이미 F1 최적점 부근에 위치

- 앙상블(스태킹 / 소프트 보팅) 미적용 근거
 - RF, LightGBM, XGBoost는 모두 동일한 SMOTE 데이터로 학습한 트리 기반 앙상블
 - ◆ 오차가 서로 상관되어 있어 결합으로 얻는 이득이 작음
 - 로지스틱 회귀는 Precision이 0.61로 낮고 확률 캘리브레이션 특성이 트리 계열과 달라 soft voting에 부적합
 - 위와 같이 RF의 개선 여지가 사실상 없어 앙상블의 기대 이득이 없다고 판단하여 적용하지 않음

- 결론
 - 목표 Recall ≥ 0.80 , F1 ≥ 0.88 , PR-AUC ≥ 0.90 을 Class 0, Class 1 모두에서 달성
 - 하이퍼파라미터 튜닝과 threshold 조정 모두 유의미한 개선을 만들지 못했으며, 이는 baseline RandomForest가 본 데이터 구성에서 이미 포화 상태임을 의미

- 한계 및 향후 개선 방향
 - test의 사기 거래가 98건에 불과함
 - ◆ 1건 오분류가 Recall 1.02%p에 해당하므로, 모델 간 소수점 셋째 자리 차이는 노이즈 범위로 해석해야 함
 - ◆ StratifiedKFold 반복 평가로 지표의 분산을 함께 보고하는 것이 바람직
 - 정상 거래 10,000건 서브샘플링의 영향
 - ◆ 실제 데이터의 클래스 비율은 약 578 : 1이나 본 실험 조건은 20 : 1
 - ◆ Precision은 클래스 비율에 직접 의존하므로 실제 운영 환경에서는 크게 하락
 - ◆ RF의 FPR = $1 / 2,001 \approx 0.0005$ 를 전체 정상 거래 284,315건에 적용하면 FP 약 142건으로 추정
 - Precision ≈ 0.75 , F1 ≈ 0.82 수준으로 하락 예상
 - FP 1건에 근거한 추정이므로 불확실성은 크나, 서브샘플링이 문제를 쉽게 만들었다는 방향성은 분명

- ◆ 따라서 본 과제의 지표는 서브샘플링된 조건에서의 값으로 한정하여 해석해야 함

- 추가로 시도할 수 있는 방법

- ◆ SMOTE 대신 `class_weight / scale_pos_weight` 기반 cost-sensitive learning 적용
- ◆ BorderlineSMOTE, SMOTE + Tomek Links 등 변형 기법 비교
- ◆ SMOTE 비율을 1 : 1이 아닌 `sampling_strategy = 0.3 ~ 0.5`로 조정
- ◆ `threshold`를 test가 아닌 OOF 예측으로 결정하여 낙관 편향 제거
- ◆ 전체 284,807건으로 평가하여 실제 운영 조건에서의 성능 확인